

Instituto Superior de Engenharia de Coimbra

Licenciatura em Engenharia Informática

Sistemas Operativos

Relatório Trabalho Prático

Programação em C para UNIX

Renato Augusto Carvalho Santos – 2019143713 LEI

João Pedro Anjo Gouveia – 2019134451 LEI

Coimbra, 14 de dezembro de 2024

Índice

1.Introdução....3

2. Descrição de ficheiros....3

3. Estruturas de dados....5

3.1. Dados do manager....5

3.2. Dados do feed....5

3.3. Dados partilhados....6

4. Arquitetura do sistema....6

4.1. Mecanismo de comunicação....7

4.2. Ciclo Select....7

4.3. Threads....8

4.4. Sincronização e proteção de dados....9

4.5. Diagrama da Arquitetura do sistema....10

5. Término dos programas....11

6. Conclusão....11

1.Introdução

Este relatório descreve a implementação do trabalho prático realizado no âmbito da unidade curricular de Sistemas Operativos. Este trabalho consiste numa plataforma de envio e receção de mensagens curtas, organizadas por tópicos. As partes utilizadas do API do sistema Unix foram abordadas nas aulas e são utilizadas funções do sistema operativo para a manipulação de recursos do sistema.

2. Descrição de ficheiros

A plataforma utiliza dois ficheiros principais, o manager.c e o feed.c. Para além disso existe um ficheiro makefile com vários targets de compilação, um ficheiro de texto de apoio ao manager e vários ficheiros .c e .h auxiliares.

- *manager.c*

O programa manager contém as definições das suas threads, duas funções principais que lidam com o stdin e com o seu pipe e um ciclo que chama essas funções.

- *feed.c*

O programa feed contém a definição da sua thread, duas funções principais que lidam com o stdin e com o seu pipe correspondente e um ciclo que chama essas funções.

- *manager_aux.c*

Contém as definições de funções auxiliares que fazem operações relativas ao manager, principalmente funções que lidam com as estruturas de dados de armazenamento do manager. Contém também as funções que lidam com o ficheiro de texto de mensagens persistentes.

- *shared.c*

Contém as definições de funções auxiliares partilhadas pelos programas manager e feed.

- *manager_aux.h*

Contém os protótipos das funções presentes no ficheiro manager_aux.c.

- *shared.h*

Contém includes, defines, um enum que define o tipo de mensagens trocadas, protótipos de funções e structs de todo o tipo de mensagens que podem ser enviadas ou recebidas pelo manager ou feed. É crucial que estas informações sejam partilhadas entre os dois programas para que a comunicação seja possível.

- *manager_data.h*

Contém as estruturas de dados utilizadas especificamente pelo manager e pelas suas threads.

- *feed_data.h*

Contém as estruturas de dados utilizadas especificamente pelo feed e as suas threads.

- *Makefile*

Este ficheiro makefile contém 4 targets de compilação, all: compilação de todos os programas, broker: compilação do programa manager, feed: compilação do programa feed e clean: remoção de todos os named pipes e ficheiros executáveis.

- *Ficheiro de Texto*

O manager requer que a variável de ambiente MSG_FICH esteja definida com um ficheiro de texto onde são recuperadas e armazenadas as mensagens persistentes. Se o ficheiro de texto definido não existir nenhuma mensagem é recuperada e no final da execução do programa manager esse ficheiro de texto é criado e as mensagens persistentes que ainda existirem são armazenadas nesse ficheiro. Se o ficheiro de texto definido existir as mensagens escritas nesse ficheiro são recuperadas e armazenadas ao iniciar o manager sendo que no término do programa as mensagens persistentes que ainda existirem são guardadas de volta.

3. Estruturas de dados

3.1. Dados do manager

No programa manager é utilizada uma struct que engloba todas as informações utilizadas e passadas entre as funções e as suas threads do tipo ManagerData. Para o armazenamento da informação dos clientes é utilizado um array de structs do tipo armazenamentoClientes e um contador de clientes. Para o armazenamento da informação dos tópicos é utilizado um array de structs do tipo armazenamentoTopicos e um contador de tópicos. No que toca ás threads existem structs com informações relativas a cada thread do tipo threadParamsDecrementa e threadParamsManagerHeartbeat.

3.2. Dados do feed

Como não é necessário armazenar informações de tópicos ou de outros clientes no programa feed são armazenadas apenas informações em duas structs relativas ao próprio feed e á sua thread. Mais uma vez é utilizada uma struct que engloba todas as suas informações para facilitar a passagem e utilização entre o programa do tipo FeedData, dentro dessa struct está também uma struct do tipo ThreadParamsManagerHeartbeat que armazena as informações relativas á sua thread.

3.3. Dados partilhados

Existe um ficheiro header (shared.h) que define todas as informações que ambos os programas necessitam de saber para poderem comunicar. Para cada mensagem que pode ser enviada ou recebida pelos processos foram criadas duas structs: uma struct que armazena as informações relativas á mensagem e uma segunda struct que serve como uma identificação do tipo de mensagem, essa struct contém a struct da informação da mensagem e um inteiro que simboliza a informação armazenada que deve ser lida em seguida. O enum utilizado no header file contém uma série de inteiros que correspondem ao tipo de mensagens que cada processo pode receber ou enviar, estes tipos são colocados no inteiro que identifica a mensagem antes de serem enviados e depois no destino o tipo é lido e a mensagem é tratada em conformidade. Desta maneira os processos possuem toda a informação necessária para se compreenderem e tratarem cada mensagem especifica de maneira correta.

4. Arquitetura do sistema

Os processos do sistema são o manager e vários feeds assim como as suas threads correspondentes. Apenas uma instância do processo manager pode e tem de estar sempre a correr para o sistema funcionar, no entanto podem existir vários processos feed a correr ao mesmo tempo, apenas um ou até nenhum a correr em simultâneo com o manager. A arquitetura foi projetada de maneira a utilizar o mínimo possível de recursos do sistema garantindo segurança e rapidez entre os processos e as suas threads.

4.1. Mecanismo de comunicação

O mecanismo utilizado para comunicação inter-processo é o named pipe. O programa manager cria e abre apenas um pipe que é depois utilizado por todos os processos feed para enviar mensagens. Cada programa feed cria e abre o seu próprio pipe para receber informações vindas do manager. O servidor (manager) comunica com os vários clientes (feeds) e cada cliente comunica com o servidor mas os clientes não são capazes de comunicar entre si.

4.2. Ciclo Select

Nas threads da função main dos programas manager e feed existe um ciclo infinito while que em cada iteração é configurada uma função select com dois file descriptors: o seu próprio pipe e o stdin. No caso de existirem dados no pipe o tipo de mensagem é lido primeiro e depois é passado esse tipo para uma função que a processa consoante o tipo, no caso de existirem dados no stdin o stdin é lido, os dados são colocados num buffer que é enviado para uma função que processa o stdin para ser tratado como um comando consoante o que foi introduzido. Desta maneira o pipe e os comandos introduzidos estão sempre a ser monitorizados ao mesmo tempo sem a necessidade de threads adicionais.

4.3. Threads

Em resumo o processo manager utiliza 3 threads e cada processo feed utiliza 2 threads contando com as threads do main. Para aumentar segurança e evitar erros foi implementado em cada programa uma thread que envia e recebe constantemente sinais de vida entre todos os processos, desta maneira o processo manager pode ter a certeza que os clientes que gere estão funcionais e a correr e cada processo feed pode ter a certeza que o processo manager ainda está a correr (processos feed não podem estar a correr sem que o manager esteja).
Em termos do processo manager são inicializadas e lançadas duas threads na função main adicionais para além da thread do main.

- **decrementaTemposDeVida**: Esta thread serve para decrementar o tempo de vida das mensagens persistentes armazenadas no manager. A cada segundo o array de tópicos é percorrido e as mensagens persistentes são verificadas: se tiverem um tempo de vida acima de zero é lhes decrementado 1 valor, se o seu tempo de vida chegou a zero essa mensagem é automaticamente removida do array de mensagens persistentes do tópico relativo. Para além disso esta thread verifica também se um tópico deve ser removido: no caso de um tópico não ter clientes subscritos e não ter mensagens persistentes esse tópico é automaticamente removido do armazenamento do manager.
- **managerHeartbeat**: A cada segundo esta thread percorre o array de clientes e verifica á quanto tempo recebeu um heartbeat de cada cliente armazenado, se qualquer cliente não der sinais de vida a algum tempo o manager assume que o processo de esse cliente deixou de correr e elimina o cliente específico no seu armazenamento. Se o manager recebeu um heartbeat do feed então envia de volta um heartbeat do manager para cada cliente armazenado pelo pipe do feed correspondente.
- **Thread do main**: Antes do main entrar num ciclo infinito primeiro inicializa os seus dados, configura um handler para um sinal SIGINT, cria a abre o seu pipe, inicializa os seus mutexes, preenche as structs que as threads utilizam, recupera as mensagens persistentes do ficheiro de texto e lança as suas duas threads. Depois desta configuração inicial o processo entra no ciclo do select.

Em termos do processo feed é inicializada e lançada uma thread adicional na função main para além da thread do main.

- **feedHeartbeat**: A cada segundo esta thread verifica á quanto tempo recebeu um heartbeat do manager, se o feed deixar de receber sinais de vida do manager a algum tempo o feed assume que o processo do manager deixou de correr e então encerra o próprio processo de forma ordeira. Caso contrário envia o seu heartbeat para o pipe do manager.
- **Thread do main**: Antes do main entrar num ciclo infinito primeiro inicializa os seus dados, configura um handler para um sinal SIGINT, abre o pipe do manager, lê o username introduzido no argumento, cria o seu pipe, envia uma mensagem do tipo LOGIN, inicializa os seus mutexes, preenche a struct que a thread utiliza e lança a thread. Tal como o programa manager depois desta configuração o processo entra no ciclo do select.

4.4. Sincronização e proteção de dados

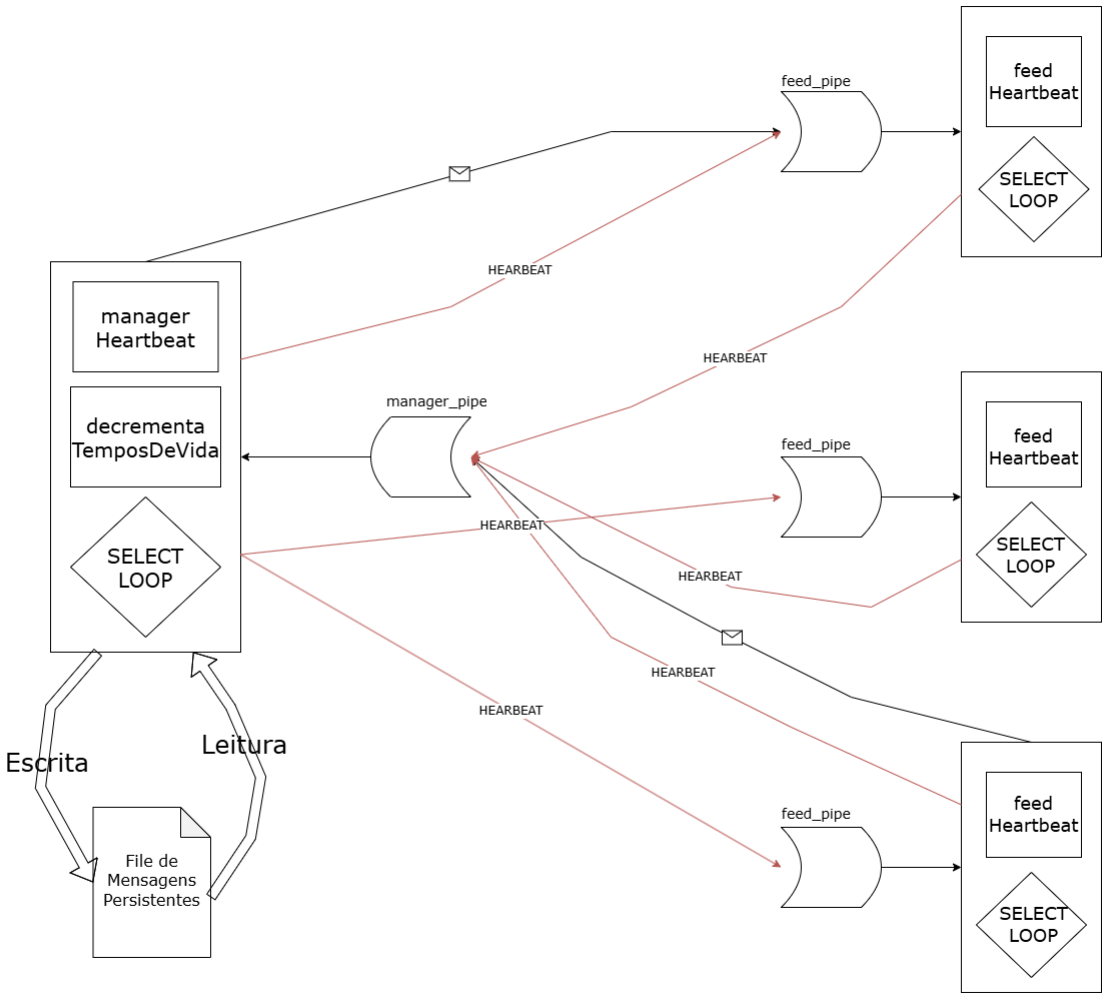
São utilizados um conjunto de mutexes nos programas feed e manager para a proteção de dados entre as threads. No caso do manager:

- **mutexStop**: protege as flags de paragem das threads.
- **mutexTime**: protege a temporização do heartbeat do manager.
- **mutexTopicos**: protege tópicos armazenados no manager.
- **mutexClientes**: protege clientes armazenados no manager.

No caso do feed:

- **mutexStop**: protege a flag de paragem das threads.
- **mutexTime**: protege a temporização do heartbeat do feed.

4.5. Diagrama da Arquitetura do sistema



5. Término dos programas

O trabalho foi desenvolvido de maneira que os processos feed e manager encerrem sempre da melhor maneira possível mesmo em situações inesperadas. Os processos terminam de forma ordeira ao sinalizarem às suas threads para terminarem (através de flags), de seguida ficam á espera que as threads terminem (através de pthread_join), destroem os seus mutexes e por último fecham o file descriptor do seu pipe e eliminam o ficheiro. No caso do manager é chamada também a função que salva as mensagens persistentes no ficheiro de texto.

- No caso de serem introduzidos os comandos respetivos de término no stdin (close para o manager e exit para o feed) os processos terminam de forma ordeira.
- No caso de receberem um sinal SIGINT terminam da mesma maneira como se tivessem recebido um comando para terminarem devido á configuração de um sinal SIGINT que chama uma função vazia que simplesmente “salta” fora do loop do select.
- No caso de serem terminados por sinais que não são configuráveis como SIGKILL ou SIGTERM não terminam de forma ordeira, mas devido às threads de heartbeat cada processo deteta que o processo foi terminado e age em conformidade.

6. Conclusão

Este trabalho prático permitiu explorar vários conceitos fundamentais sobre a programação em C para sistemas UNIX, com foco em comunicação inter-processo e gestão de recursos através de named pipes. O desenvolvimento da plataforma apresentou desafios, principalmente na sincronização entre processos e threads, e a proteção de dados partilhados através da utilização de mutexes garantindo a segurança e consistência dos dados. A implementação das threads de heartbeat revelou-se essencial para monitorizar o estado dos processos e assegurar uma terminação ordeira em caso de falhas ou interrupções. O sistema resultante demonstrou ser eficiente e funcional, atingindo os objetivos propostos.